
WetterExtended

Benutzerhandbuch

Radar-basiertes Sturmwzellen-Tracking-System für Kärnten/Österreich

Zielgebiet: Klagenfurt · Villach · Wolfsberg · Spittal · St. Veit

Dokumentversion	v1.1 (Ergänzung Mai 2026)
Basisversion	v1.0
Letzte Änderung	Mai 2026
Sprache	Deutsch
Zielpattform	Raspberry Pi 5 B · Hailo-8 AI · 16 GB · 26 TOPS
Betriebssystem	Raspberry Pi OS Bookworm 64-bit (Kernel ≥ 6.6)
Repository	github.com/IVOBLA/WetterExtended (Branch: main)
Lokaler Pfad	/home/ki-pi/wetterprojekt

Dieses Dokument ist das offizielle Benutzerhandbuch für das WetterExtended-System. Abschnitt 9 ff. dokumentieren alle Erweiterungen, die nach Version v1.0 eingeführt wurden.

Inhaltsverzeichnis

1 Systemübersicht

2 Voraussetzungen und Installation

- 2.1 install.sh — Vollinstallation (--mode=full)
- 2.2 install.sh — Upgrade (--mode=upgrade)
- 2.3 install.sh — Optionen

3 Betrieb und systemd-Services

4 Admin-Panel — Seiten und Bedienung

- 4.1 Dashboard
- 4.2 Karte (MapView)
- 4.3 Live-Daten
- 4.4 Datensatz
- 4.5 Atmosphäre
- 4.6 Orte
- 4.7 Schwellwerte
- 4.8 Horizonte
- 4.9 Training
- 4.10 Konfiguration
- 4.11 Modell-Fortschritt
- 4.12 Genauigkeit / Accuracy
- 4.13 Logs
- 4.14 KI-Analyse
- 4.15 Vollbild-Karte (/karte)

5 Externe Datenquellen und APIs

6 ML-Pipeline

- 6.1 Modelle
- 6.2 Features
- 6.3 Training-Zeitplan

7 Vorhersage-Verifikation (Closed-Loop)

8 KMZ-Export

9 NEU: Erweiterungen nach v1.0 — Übersicht [\[NEU\]](#)

10 NEU: Optical Flow (pysteps Lucas-Kanade) [\[NEU\]](#)

11 NEU: AROME icon_d2 Gitterpunkt-Wetterdaten [\[NEU\]](#)

12 NEU: Erweiterte Geländemodell-Features (DEM) [\[NEU\]](#)

13 NEU: Windscherung und Hagelindikator [\[NEU\]](#)

14 NEU: Atmosphären-Seite und Großwetterlage [\[NEU\]](#)

15 NEU: GeoSphere TAWES und Nowcast-Daten [\[NEU\]](#)

16 NEU: SMS-Benachrichtigungen (Twilio) [\[NEU\]](#)

17 NEU: Daten-Rotation (cleanup_old_data.py) [\[NEU\]](#)

- 18 NEU: Disk-Monitoring im Dashboard** [\[NEU\]](#)
- 19 NEU: Admin-Panel Authentifizierung (nginx)** [\[NEU\]](#)
- 20 NEU: Adaptiver Loop-Intervall** [\[NEU\]](#)
- 21 NEU: LOCAL_TRAINING-Flag** [\[NEU\]](#)
- 22 NEU: API-Request-Statistik** [\[NEU\]](#)
- 23 NEU: KI-Analyse Chat (daily_analyzer.py)** [\[NEU\]](#)
- 24 Hailo-8 Integration — Phasen-Roadmap**
- 25 Konfigurationsreferenz**
- 26 Fehlerbehebung (Troubleshooting)**
- 27 Änderungshistorie**

1 Systemübersicht

WetterExtended ist ein lokales Radar-basiertes Sturmwzellen-Tracking- und Nowcasting-System für das Gebiet Kärnten/Österreich. Es verarbeitet ARSO INCA si0zm Radar-KMZ-Bilder in einem 5-Minuten-Takt, erkennt und verfolgt Niederschlagszellen mittels HSV-Farbsegmentierung und Kalman-Filter, und berechnet kurzzeitige Bewegungsvorhersagen (Nowcasting) mit mehreren Maschinellen-Lern-Modellen.

Schicht	Komponente	Beschreibung
Datenbeschaffung	radar_download.py blitz_api.py weather_api.py fetch_arome_openmeteo.py fetch_700hpa_wind_per_object_slim.py fetch_geosphere_nowcast.py	ARSO KMZ, Blitzortung, GeoSphere TAWES, Open-Meteo AROME icon_d2, 700 hPa Höhenwind, Nowcast-Daten
Verarbeitung	object_tracking.py optical_flow_features.py orographic_module.py assign_cape_from_forecast.py cloud_height_from_eumetview.py	HSV-Segmentierung, Kalman-Tracking, Optical Flow, DEM-Features, CAPE, Wolkenhöhe EUMETView
Vorhersage	prediction.py model_training.py radar_convlstm.py	LSTM + LightGBM (5 Horizonte), ConvLSTM, kinetisches Fallback
Persistenz	accuracy_tracker.py cleanup_old_data.py runtime_config.py	Closed-Loop-Verifikation, Daten-Rotation (90 Tage), Runtime-Override-System
Backend-API	app.py (Flask)	JSON-REST-API für Frontend, >30 Endpunkte
Frontend	React/Vite (15 Seiten) nginx Reverse-Proxy	Admin-Panel, Leaflet-Karte, Basic-Auth über nginx
Infrastruktur	install.sh scheduler.py sms_notifier.py daily_analyzer.py	Automatisiertes Setup, Cron-Jobs, SMS-Warnungen, KI-gestützte Analyse
KI-Beschleuniger	hailo_inference.py (Phase B)	Hailo-8 (26 TOPS) für U-Net Nowcasting (geplant Phase B)

■ Hinweis: Hailo-8 Inferenz ist in Phase A als Stub implementiert. Die vollständige Integration erfolgt in Phase B nach Anschaffung eines x86-Linux-Trainers.

1.1 systemd-Services

Service	Datei	Funktion
wetterprojekt.service	main.py	Live-Loop (Radar → Tracking → Vorhersage → Upload)
wetterprojekt-scheduler.service	scheduler.py	Cron-Jobs (Training, Cleanup, Genauigkeit, KI-Analyse)
wetterprojekt-admin.service	app.py	Flask REST-API (Port 5000, erreichbar via nginx Port 80)

2 Voraussetzungen und Installation

Die Installation erfolgt ausschließlich über das Skript **install.sh**. Es erkennt den Systemzustand automatisch und führt alle notwendigen Schritte aus.

2.1 Vollinstallation (--mode=full)

■ **Achtung: LÖSCHT alle Trainingsmodelle, Radar-Daten und Objekt-Historien! Das Copernicus DEM, .env und runtime_overrides.json bleiben erhalten.**

```
bash install.sh --mode=full --repo git@github.com:IVOBLA/WetterExtended.git
```

Die Vollinstallation führt folgende Phasen durch:

- Prüfung OS-Version (Bookworm 64-bit), Kernel (≥ 6.6), Festplatte (min. 4 GB frei), RAM
- Git-Clone/Pull des Repositories
- System-Pakete: python3, pip, nginx, apache2-utils, ffmpeg, libhdf5-dev usw.
- Hailo-APT-Repository + hailo-all Treiber-Installation
- Python-venv erstellen + pip-Pakete installieren
- Copernicus DEM herunterladen (einmaliger Großdownload, ca. 800 MB)
- Node.js + npm installieren, Frontend bauen (npm install && npm run build)
- nginx Reverse-Proxy konfigurieren + Basic-Auth-Passwort generieren
- systemd-Services anlegen und aktivieren
- .env-Datei anlegen (FTP, Blitzortung, Twilio-Zugangsdaten)

2.2 Upgrade (--mode=upgrade)

■ Aktualisiert nur den Source-Code. Modelle, Trainingsdaten und .env bleiben unverändert.

```
bash install.sh --mode=upgrade
```

Beim Upgrade werden aktualisiert: Python-Pakete, Frontend-Build, systemd-Unit-Dateien.

2.3 install.sh — Optionen

Option	Standard	Beschreibung
--mode=full upgrade	upgrade	Installations-Modus (full = Neuinstallation)
--repo URL	—	Git-Repository-URL (SSH oder HTTPS)
--branch NAME	main	Git-Branch
--target PFAD	/home/ki-pi/wetterprojekt	Zielpfad auf dem Pi
--no-hailo	—	Hailo-Installation überspringen
--no-node	—	Node.js/Frontend überspringen
--no-training	—	LOCAL_TRAINING=false setzen
--enable-services	—	systemd-Services nach Installation starten
--local	—	Lokale Dateien statt Git-Clone verwenden

■ **Hinweis:** Das generierte Admin-Passwort wird am Ende der Installation angezeigt und in `.admin_password` (Modus 600) gespeichert.

3 Betrieb und systemd-Services

Aktion	Befehl
Status aller Services	<code>sudo systemctl status wetterprojekt wetterprojekt-scheduler wetterprojekt-admin</code>
Live-Loop starten/stoppen	<code>sudo systemctl start stop wetterprojekt.service</code>
Scheduler starten/stoppen	<code>sudo systemctl start stop wetterprojekt-scheduler.service</code>
Admin-Panel starten/stoppen	<code>sudo systemctl start stop wetterprojekt-admin.service</code>
Logs verfolgen (Live-Loop)	<code>journalctl -fu wetterprojekt.service</code>
Logs verfolgen (Scheduler)	<code>journalctl -fu wetterprojekt-scheduler.service</code>
Services nach Update neu laden	<code>sudo systemctl daemon-reload && sudo systemctl restart wetterprojekt wetterprojekt-scheduler</code>

Das Admin-Panel ist erreichbar unter **`http://<PI-IP>/`** (Port 80, nginx, Basic-Auth). Die öffentliche Vollbild-Karte ist ohne Authentifizierung unter **`http://<PI-IP>/karte`** erreichbar.

4 Admin-Panel — Seiten und Bedienung

Das Admin-Panel ist eine React-Applikation (Vite + React 18 + Tailwind CSS + Leaflet + Recharts) mit 15 Seiten. Es wird über nginx auf Port 80 ausgeliefert und kommuniziert mit dem Flask-Backend (Port 5000) über eine REST-JSON-API.

4.1 Dashboard (/)

Systemstatus auf einen Blick: Anzahl aktiver Zellen, letzter Radar-Zeitstempel, Modell-Status, API-Gesundheit, **Disk-Nutzung mit Farbampel** (grün/gelb/rot), RAM-Auslastung.

4.2 Karte (/map)

Interaktive Leaflet-Karte mit Sturmwellen (Kontur + ID-Label), Vorhersage-Pfeilen (farbcodiert nach Horizont), Ortsdurchquerungs-Markierungen, Bewegungspfad-Historie, **Hagelwarnungs-Rahmen** (rot) und **Stationär-Marker** (⊕ amber). Farblegende unten links.

4.3 Live-Daten (/live)

Tabelle aller aktuell erkannten Zellen mit allen ML-Features: Position, Geschwindigkeit, CAPE, Wolkenhöhe, Blitze, Optical-Flow, AROME-Werte, Hagelwahrscheinlichkeit, Windscherung.

4.4 Datensatz (/data)

Statistiken über die gesammelten Trainingsdaten: Anzahl Samples pro Horizont, Feature-Vollständigkeit, letztes Dataset-Rebuild.

4.5 Atmosphäre (/atmosphaere) [NEU]

Großwetterlage-Snapshot für Kärnten: 500-hPa-Geopotential, Steuerströmung, AROME-Gitterpunktwerte (T, Taupunkt, Windböen, Lifted Index, Gefriergrenze), stratiforme Niederschlagsumgebung. Wird alle 30 Minuten aktualisiert.

4.6 Orte (/locations)

Definition von Überwachungsorten mit Umkreis (km). Durchquerungsanzeige in der Farbe des jeweiligen Vorhersage-Horizonts.

4.7 Schwellwerte (/thresholds)

HSV-Farbschwellwerte für die Zellerkennung (Gewitterzellen, moderate Zellen, Minimum-Intensität). Änderungen wirken sofort auf den nächsten Live-Loop-Zyklus.

4.8 Horizonte (/horizons)

Konfiguration der 5 Vorhersage-Zeiträume (default: 10/20/30/40/60 min) und deren Pfeifarben.

4.9 Training (/training)

Manueller Trainings-Trigger für alle Modelle, Anzeige des letzten Trainingszeitstempels und des LOCAL_TRAINING-Status.

4.10 Konfiguration (/config)

Vollständige JSON-Konfiguration direkt editieren (fortgeschrittene Benutzer). Vorsicht: fehlerhafte JSON bricht die Konfiguration.

4.11 Modell-Fortschritt (/progress)

Lernkurven (Loss über Epochen) für LSTM und ConvLSTM als Recharts-Grafik.

4.12 Genauigkeit (/accuracy)

Closed-Loop-Verifikation: MAE, Hit-Rate und durchschnittliche Abweichung pro Horizont über einstellbaren Zeitraum.

4.13 Logs (/logs)

Anzeige der letzten N Zeilen des System-Logs, API-Fehler-Log, API-Request-Zähler pro Tag und externer Schnittstelle.

4.14 KI-Analyse (/ai-analysis) [NEU]

Tägliche automatische KI-Analyse des Systemzustands via Claude API. Zeigt priorisierte Handlungsempfehlungen (CRITICAL/HIGH/MEDIUM/LOW). Interaktiver Chat-Bereich für Fragen an die KI.

4.15 Vollbild-Karte (/karte) [NEU]

Öffentliche Seite ohne Admin-Authentifizierung. Zeigt nur die Leaflet-Karte im Vollbild — geeignet für Einbindung auf externen Displays oder Websites.

5 Externe Datenquellen und APIs

API / Dienst	URL	Update-Intervall	TTL Cache	Bemerkung
ARSO INCA si0zm Radar	ameteo.arso.gov.si/...	5 Min	If-Modified-Since	KMZ, öffentlich, kein Key
Open-Meteo AROME icon	open-meteo.com/v1/forecast	3 h (Modell-Run)	30 Min	10000 Req/Tag (Free). Bulk-Query!
Open-Meteo 700 hPa / 500 hPa	open-meteo.com/v1/forecast	6 h (Modell-Run)	60 Min	Höhenwind, Großwetterlage
GeoSphere CAPE	dataset.api.hub.geosphere.at/v1/forecast/nwp	30 Min	2500m	Österreichischer WD, kein Key
GeoSphere TAWES	dataset.api.hub.geosphere.at/v1/station/current/tawes	10 Min	10 Min	Stationsdaten Kärnten (31 Stationen)
GeoSphere Nowcast	dataset.api.hub.geosphere.at/v1/series/forecast/nowcast-v1	15 Min	15 Min	Niederschlag- und Böen-Nowcast
EUMETView MSG IR108	view.eumetsat.int/geoserver/...	15 Min	10 Min	Satellit-WMS für Wolkenhöhe (TIFF)
Blitzortung	data.blitzortung.org/Data/Protected	10 Min	60 s	Account nötig, HTTP Basic-Auth
Copernicus DEM 30m	copernicus-dem-30m.s3.amazonaws.com	einmalig	lokal gecacht	3 Kacheln (E013/E014/E015), ~800 MB
Claude API (Anthropic)	api.anthropic.com	täglich 06:00	—	KI-Analyse, nur wenn aktiviert

■ **Hinweis:** Alle API-TTLs sind über `runtime_overrides.json` überschreibbar. Die Request-Zähler pro Tag und Schnittstelle sind im Admin-Panel unter Logs sichtbar.

■ **Achtung:** lightningmaps.org wird NICHT verwendet (inoffiziell). Blitzdaten kommen ausschließlich über Blitzortung.org (HTTP Basic-Auth).

6 ML-Pipeline

6.1 Modelle

Modell	Datei	Aufgabe	Horizonte
LightGBM (Punkt)	model_training.py	Mittelpunkt-Vorhersage (lat/lon)	10/20/30/40/60 min
LightGBM (Quantile)	model_training.py	Unsicherheits-Ellipsen (q10/q90)	10/20/30/40/60 min
LSTM	model_training.py	Zeitreihen-basierte Punkt-Vorhersage	10/20/30/40/60 min
ConvLSTM	radar_convlstm.py	Pixel-basierte Radarbild-Vorhersage	1 Horizont
U-Net (geplant)	hailo_inference.py	Vollständige Radarbild-Nowcasting	Phase B

Alle Vorhersagen erfolgen als **Regression** (Zielwert = Koordinate in Grad). Liegt kein trainiertes Modell vor, wird automatisch ein kinematisches Fallback (letzte bekannte Geschwindigkeit) verwendet.

6.2 Features (vollständige Liste)

Das ML-Modell verwendet folgende Feature-Gruppen. Gesamtanzahl wird automatisch aus config.py berechnet (ML_NUM_FEATURES).

Feature-Gruppe	Features	Quelle
Zellgeometrie & Kinematik	x, y, vx, vy, size, area, eccentricity, core_ratio, kenna	Kenna-Tracker
Höhenwind 700 hPa	wind_speed_700hPa, wind_dir_cos, wind_dir_sin	Open-Meteo icon_global
Thermodynamik	cape, cloud_top_height_msl	GeoSphere AROME / EUMETView
Topographie (Copernicus DEM)	dem_elevation_m, dem_slope_toward_cell, dem_slopeside_m	Copernicus DEM 30m
Blitze	lightning_count_10km	Blitzortung.org
Optical Flow [NEU]	of_vx, of_vy, of_speed, of_divergence	pysteps Lucas-Kanade
AROME icon_d2 Gitterpunkte [NEU]	arome_t2m, arome_td2m, arome_ff10m, arome_hl10m, arome_d2_sin, arome_li, arome_fl_height	Open-Meteo icon_d2
Stratiforme Umgebung [NEU]	strat_area_px, strat_intensity_mean, strat_dbz_h50m	GIS-Segmentierung
Talkanalisisierung [NEU]	valley_alignment, valley_distance_km, valley_configuration	Orographisches Modul
Großwetterlage 500 hPa [NEU]	500_dam, wind_500_speed, wind_500_dir_cos, wind_500_dir_sin	Open-Meteo icon_global
Orographische Risk-Scores [NEU]	hail_blocking_score, orographic_lift_score, orographic_risk	Orographisches Modul
Windscherung [NEU]	wind_shear_speed, wind_shear_dir_cos, wind_shear_dir_sin	Open-Meteo (10m + 700 hPa)
Hagelindikator [NEU]	hail_prob	Heuristik (CAPE + Schichtung + Blitze)
GeoSphere-Nowcast [NEU]	nowcast_rr_mm15, nowcast_ffx_kmh, nowcast_hail	GeoSphere Nowcast API
Stationsdaten [NEU]	RR, DD, FF, FFX, GLOW, P, RF, TL, TP	GeoSphere TAWES (31 Stationen)
Zeit	hour_sin, hour_cos, month_sin, month_cos	Systemzeit

■ [NEU] = nach v1.0 eingeführt. Bei Feature-Änderungen müssen Modelle neu trainiert werden.

6.3 Training-Zeitplan

Job	Standard-Zeitplan	Konfigurierbar via
Dataset-Rebuild	stündlich	DATASET_REBUILD_INTERVAL_MIN
LightGBM/LSTM Retrain (Intervall)	alle 6 h	retrain_interval_hours
LightGBM/LSTM Retrain (Nightly)	täglich 03:00	retrain_cron_hour / retrain_cron_minute
ConvLSTM Retrain	montags 02:00	convlstm_cron_*
Genauigkeits-Evaluation	stündlich	—
KI-Analyse	täglich 06:00	AI_ANALYSIS_CONFIG.cron_hour
Daten-Cleanup	täglich 04:30	DATA_CLEANUP_CRON_HOUR/MINUTE

7 Vorhersage-Verifikation (Closed-Loop)

Das System vergleicht nach Ablauf jedes Vorhersage-Horizonts die vorhergesagte Position mit der tatsächlich beobachteten. Die Ergebnisse werden in `train_data/evaluation/` als JSONL gespeichert.

Parameter	Wert	Bedeutung
VERIFICATION_TOLERANCE_KM	5 km	Treffer wenn tatsächliche Zelle ≤ 5 km von Vorhersage
VERIFICATION_TIME_TOLERANCE_S	90 s	Zeitfenster für Frame-Suche (ARSO liefert alle 2–5 min)
VERIFICATION_MAX_SEARCH_RADIUS_KM	25 km	Suchradius für Nearest-Neighbor-Match

Im Admin-Panel unter **Genauigkeit** ist die durchschnittliche Abweichung (MAE) und Hit-Rate pro Horizont über einen einstellbaren Zeitraum grafisch dargestellt.

8 KMZ-Export

Nach jedem Live-Loop-Zyklus wird eine **forecast.kmz**-Datei erzeugt und per FTP hochgeladen. Sie kann direkt in Google Earth, OziExplorer oder kompatible Kartensoftware importiert werden.

- Aktuell erkannte Zellen als Polygon-Konturen
- Vorhersage-Pfeile pro Horizont (farbcodiert)
- Unsicherheits-Ellipsen (q10/q90) um Vorhersagepunkte
- Ortdurchquerungsmarkierungen

■ *Hinweis: Die KMZ enthält ausschließlich Vorhersage-Daten des aktuellen Zyklus. Historische Daten sind nicht enthalten.*

9 NEU: Erweiterungen nach v1.0 — Übersicht

Die folgende Tabelle fasst alle nach Version v1.0 eingeführten Erweiterungen zusammen. Die nachfolgenden Kapitel 10–23 beschreiben jede Erweiterung im Detail.

Kapitel	Erweiterung	Datei(en)	Phase
10	Optical Flow (pysteps Lucas-Kanade)	optical_flow_features.py	A
11	AROME icon_d2 Gitterpunkt-Wetterdaten	fetch_arome_openmeteo.py	A
12	Erweitertes DEM (3 Kacheln, Talkanalisis, 500m-Data)	dem.py, fetch_synoptic_features.py	A
13	Windscherung + Hagelindikator	main.py, config.py	A
14	Atmosphären-Seite + Großwetterlage 500m-Isophäre	500m_isophare.jsx, fetch_synoptic_features.py	A
15	GeoSphere TAWES + Nowcast-Daten	fetch_geosphere_nowcast.py, fetch_tawes_gust.py	A
16	SMS-Benachrichtigungen (Twilio)	sms_notifier.py, main.py	A
17	Daten-Rotation / Cleanup	cleanup_old_data.py, scheduler.py	A
18	Disk-Monitoring im Dashboard	app.py, Dashboard.jsx	A
19	Admin-Panel Authentifizierung (nginx)	install.sh	A
20	Adaptiver Loop-Intervall	main.py, config.py	A
21	LOCAL_TRAINING-Flag	config.py, scheduler.py, install.sh	A
22	API-Request-Statistik	app.py, Logs.jsx	A
23	KI-Analyse Chat (daily_analyzer.py)	daily_analyzer.py, AiSuggestions.jsx	A

10 NEU: Optical Flow (pysteps Lucas-Kanade)

Modul: **optical_flow_features.py** | Bibliothek: pysteps

Zwischen zwei aufeinanderfolgenden Radarbildern wird mit dem Lucas-Kanade-Algorithmus ein dichtes Optical-Flow-Feld berechnet. Jedem erkannten Sturmzell-Objekt wird der lokale Bewegungsvektor am Zell-Mittelpunkt zugewiesen. Dies ermöglicht dem ML-Modell zu erkennen, ob sich ein Niederschlagsfeld stärker oder schwächer bewegt als der Kalman-Schätzer, und ob das Feld lokal divergiert (Auseinanderfall) oder konvergiert (Intensivierung).

Feature	Einheit	Bedeutung
of_vx	px/Frame	Flow-Vektor x am Objektzentrum (auf Original-Pixeleinheiten skaliert)
of_vy	px/Frame	Flow-Vektor y am Objektzentrum
of_speed	px/Frame	Betrag des Flow-Vektors (Betrag aus vx/vy)
of_divergence	1/Frame	Lokale Divergenz ($\partial u/\partial x + \partial v/\partial y$), pos. = divergent
of_available	0/1	Fallback-Flag: 0 = kein Flow verfügbar (fehlendes Bild)

■ Hinweis: Fällt pysteps nicht installiert oder ein Bild fehlt, werden alle OF-Features auf 0 gesetzt (of_available=0). Das Modell lernt dies als 'fehlend' und verliert keine Trainingsdaten.

11 NEU: AROME icon_d2 Gitterpunkt-Wetterdaten

Modul: **fetch_arome_openmeteo.py** | API: Open-Meteo icon_d2 (2,2 km Gitter)

Für jede erkannte Sturmzelle werden AROME-Gitterpunkt-Wetterdaten direkt auf dem Zell-Mittelpunkt abgerufen. Um das API-Limit (10.000 Requests/Tag) nicht zu überschreiten, werden ALLE Zellen in einem einzigen Bulk-Request abgefragt (komma-separierte Koordinaten). Der Cache-TTL beträgt 30 Minuten.

Feature	Einheit	Beschreibung
arome_t2m	°C	Temperatur 2 m über Grund
arome_td2m	°C	Taupunkt 2 m (Feuchtigkeitsindikator)
arome_ff10m	km/h	Windgeschwindigkeit 10 m
arome_dd_cos	—	cos(Windrichtung 10 m) — zyklisch kodiert
arome_dd_sin	—	sin(Windrichtung 10 m) — zyklisch kodiert
arome_li	°C	Lifted Index (neg. = konvektiv instabil)
arome_fl_height	m	Gefriergrenze über MSL

■ Der Lifted Index ist ein wichtiger Konvektions-Indikator: Werte unter -4°C signalisieren starke Gewitterneigung.

12 NEU: Erweiterte Geländemodell-Features (DEM)

Modul: **orographic_module.py** | Datenquelle: Copernicus DEM 30m

Das Geländemodell deckt ganz Kärnten mit 3 Kacheln ab (E013N46, E014N46, E015N46). Neben den bisherigen DEM-Basis-Features wurden folgende orographische Features neu eingeführt, die für die charakteristische Gewitter-Kanalisation in Kärntner Tälern (Drautal, Lavanttal, Gailtal) besonders relevant sind:

Feature	Beschreibung
dem_barrier_ahead	Höhendifferenz 10–20 km voraus in Bewegungsrichtung (m, positiv = Barriere). Zeigt ob die Zelle auf einer Barriere steht.
valley_alignment	$ \cos(\text{Winkel}) $ zwischen Zellbewegung und Talachse (0–1). 1 = Zelle bewegt sich talentlang, 0 = Zelle bewegt sich quer zum Tal.
valley_distance_km	Abstand der Zelle von der nächstgelegenen Talmitte (km).
valley_confinement	1.0 wenn Zelle im Talquerschnitt liegt, sonst 0.0.
terrain_blocking_score	0–1: Gelände blockiert wahrscheinlich die Zugbahn.
orographic_lift_score	0–1: Staulage / orographische Hebung an der Zellposition.
stationary_risk	0–1: Gesamtrisiko dass die Zelle stationär bleibt (Kombination aus Blockierung, Talkanalisation, CA).

■ Hinweis: Das Copernicus DEM wird nur einmalig beim Full-Install heruntergeladen (~800 MB) und lokal gecacht. Es wird bei Upgrades NICHT gelöscht.

13 NEU: Windscherung und Hagelindikator

13.1 Windscherung

Die vertikale Windscherung wird als Differenzvektor zwischen dem 10-m-Bodenwind (AROME icon_d2) und dem 700-hPa-Höhenwind (Open-Meteo) berechnet:

Feature	Einheit	Beschreibung
wind_shear_speed	km/h	Betrag des Scherungsvektors (10m→700hPa)
wind_shear_dir_cos	—	cos(Richtung des Scherungsvektors)
wind_shear_dir_sin	—	sin(Richtung des Scherungsvektors)

■ Hohe Windscherung (> 30 km/h über 700 m) ist ein wichtiger Indikator für organisierte Konvektion und Superzellen-Potenzial.

13.2 Hagelindikator (hail_prob)

Die Hagelwahrscheinlichkeit wird pro Zelle heuristisch aus folgenden Faktoren berechnet (kombinierte Gewichtung, Ergebnis 0.0–1.0):

- CAPE > 1000 J/kg (hohe Aufstiegsenergie)
- Lifted Index < -3°C (Instabilität der Atmosphäre)
- Blitze > 3 in 10 km (aktive Konvektion)
- Gefriergrenze < 3500 m (Hagel erreicht Boden)
- Windscherung > 20 km/h (Hagel-Umlaufstrom)
- Zellgröße und Core-Ratio (größere Kerne = höheres Potenzial)

Parameter	Standard	Beschreibung
HAIL_WARN_THRESHOLD	0.45	Auslösung Hagelwarnung wenn hail_prob ≥ Wert
STATIONARY_RISK_MARKER_THRESHOLD	0.60	Anzeige ⊕-Marker wenn stationary_risk ≥ Wert
GUST_WARN_KMH	60 km/h	Sturmböenwarnung ab diesem Wert (Nowcast oder TAWES)
HEAVY_RAIN_WARN_MM_PER_H	25 mm/h	Starkregenwarnungs-Schwellwert

In der Leaflet-Karte erscheint bei **Hagelwarnung** ein roter gestrichelter Rahmen um die betroffene Zelle mit dem Prozentwert. Bei **Stationärrisiko** erscheint ein ⊕-Symbol in Amber-Farbe.

14 NEU: Atmosphären-Seite und Großwetterlage 500 hPa

Seite: `/atmosphaere` | Modul: `fetch_synoptic_features.py`

Die neue Atmosphären-Seite zeigt einen aktuellen Großwetterlage-Snapshot für Kärnten. Daten werden alle 30 Minuten aktualisiert (ATMOSPHERIC_SNAPSHOT_INTERVAL_MIN = 30), unabhängig davon ob Sturmzellen erkannt wurden.

Anzeige-Karte	Datenquelle	Beschreibung
500-hPa-Geopotential (dam)	Open-Meteo icon_global	Steuerströmung für Großwetterlage über Mitteleuropa
Steuerströmungsvektor 500 hPa	Open-Meteo icon_global	Windgeschwindigkeit und -richtung auf 500 hPa
AROME Bodenfelder	Open-Meteo icon_d2	T2m, Taupunkt, Windgeschwindigkeit, Böen
Lifted Index	Open-Meteo icon_d2	Instabilitäts-Indikator für Konvektion
Gefriergrenze	Open-Meteo icon_d2	Höhe der 0°C-Linie über MSL
Stratiforme Niederschlagsumgebung	ML-Segmentierung	Fläche und mittlere Intensität des Stratiform-Echobands

15 NEU: GeoSphere TAWES und Nowcast-Daten

15.1 TAWES (Stationsdaten)

Modul: `fetch_tawes_gust.py` | API: GeoSphere TAWES v1 (tawes-v1-10min)

Von 31 Kärntner Wetterstationen werden alle 10 Minuten folgende Parameter abgefragt: RR (Niederschlag), DD/FF/FFX (Wind/Böen), P (Luftdruck), RF (Feuchte), TL/TP (Temperatur/Taupunkt). Dem Live-Loop wird über `max_gust_near()` der maximale gemessene Böenwert im 30-km-Umkreis jeder Zelle zugewiesen (tawes_max_gust_kmh).

15.2 GeoSphere Nowcast

Modul: `fetch_geosphere_nowcast.py` | API: nowcast-v1-15min-1km

Der GeoSphere-Nowcast liefert kurzzeitige Vorhersagen für Niederschlag (mm/15min), Böen (km/h) und stündlichen Niederschlag pro Gitterpunkt. Diese werden als ML-Features eingesetzt (nowcast_rr_mm15, nowcast_ffx_kmh, nowcast_rain_rate_1h) und für die Böenwarnung herangezogen.

■ Hinweis: Cache-TTL Nowcast: 12 Minuten (Intervall 15 Min). Cache-TTL TAWES: 10 Minuten (Intervall 10 Min).

16 NEU: SMS-Benachrichtigungen (Twilio)

Modul: **sms_notifier.py** | Dienst: Twilio (kostenpflichtiger Account erforderlich)

Bei Ortsdurchquerungen durch Sturmzellen wird automatisch eine SMS an konfigurierte Nummern gesendet. Außerdem werden Entwarnungen gesendet, wenn eine Zelle den Umkreis eines Ortes verlässt.

Ereignis	SMS-Inhalt
Ortsdurchquerung erkannt	WARNUNG: Zelle {ID} durchquert {Ort} in {Horizont} min (Farbe: {Farbe})
Entwarnung	ENTWARNUNG: Kein Gewitter mehr in Richtung {Ort}.

Konfiguration in **.env**:

```
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxx TWILIO_AUTH_TOKEN=xxxxxxxxxxxxxxxxx  
TWILIO_FROM=+43xxxxxxxxx TWILIO_TO=+43xxxxxxxxx
```

■ Hinweis: SMS-Versand ist standardmäßig deaktiviert und muss über die **.env** aktiviert werden. Schlägt der SMS-Versand fehl, wird dies geloggt aber der Live-Loop nicht unterbrochen.

17 NEU: Daten-Rotation (cleanup_old_data.py)

Modul: **cleanup_old_data.py** | Scheduler-Job: täglich 04:30

Ohne Daten-Rotation würden die Trainingsdaten unbegrenzt wachsen und bei intensivem Wetterbetrieb (viele Zellen, kurze Intervalle) nach einigen Monaten die SD-Karte des Raspberry Pi füllen. Der Cleanup-Job löscht alle Dateien die älter als **DATA_RETENTION_DAYS** (default: 90 Tage) sind.

Parameter	Standard	Beschreibung
DATA_RETENTION_DAYS	90	Maximales Datenalter in Tagen
DATA_CLEANUP_CRON_HOUR	4	Uhrzeit des täglichen Cleanup-Jobs
DATA_CLEANUP_CRON_MINUTE	30	Minute des täglichen Cleanup-Jobs

Folgende Verzeichnisse werden rotiert:

- train_data/radar/
- train_data/objects/
- train_data/weather/
- train_data/wind/
- train_data/cape/
- train_data/lightning/
- train_data/ir/
- train_data/ir_cells/
- train_data/cloud/
- train_data/arome/

■ Folgende Verzeichnisse werden NICHT rotiert: train_data/models/ (Modell-Versionierung), train_data/evaluation/ (Genauigkeits-Historie), train_data/dataset/ (aktuelles Dataset), train_data/dem/ (Copernicus DEM, großer Einmal-Download).

18 NEU: Disk-Monitoring im Dashboard

API-Endpunkt: **GET /api/disk** | Frontend: **Dashboard.jsx**

Das Dashboard zeigt eine Statusampel für die Festplattenbelegung:

Farbe	Zustand	Auslösung
Grün	Normal	Belegung < 70%
Gelb	Warnung	Belegung 70–90%
Rot	Kritisch	Belegung > 90%

Zusätzlich wird der RAM-Auslastungs-Prozentsatz angezeigt (aus `/proc/meminfo`). Bei kritischer Disk-Belegung erscheint ein farbiger Warnbanner im Dashboard-Header.

19 NEU: Admin-Panel Authentifizierung (nginx)

Das Admin-Panel ist seit dieser Version durch **HTTP Basic-Auth** geschützt. Die Authentifizierung erfolgt über nginx (nicht über Flask).

Detail	Wert / Beschreibung
Benutzername	admin (fest)
Passwort	Zufällig generiert bei Erstinstallation (openssl rand)
Passwort-Speicherort	.admin_password im Projektverzeichnis (Modus 600)
htpasswd-Datei	/etc/nginx/.htpasswd
Wiederherstellung	Beim nächsten <code>install.sh --mode=upgrade</code> wird .htpasswd aus .admin_password wiederhergestellt

Folgende Endpunkte sind OHNE Authentifizierung erreichbar:

- /karte — Vollbild-Karte (nur lesende API-Endpunkte)
- /assets/ — Frontend-Assets (werden von /karte benötigt)
- /api/objects, /api/forecast, /api/locations, /api/horizons, /api/health, /api/radar_image, /api/radar_bounds, /api/radar_timing

■ **Achtung: Das Admin-Passwort wird bei der Erstinstallation einmalig im Terminal angezeigt. Notiere es sofort! Es wird in .admin_password im Projektverzeichnis gespeichert.**

20 NEU: Adaptiver Loop-Intervall

Datei: **main.py**, **config.py**

Der Live-Loop passt seinen Schlaf-Intervall automatisch an die aktuelle Wetter-Situation an. Dies spart Rechenleistung und API-Requests in ruhigen Phasen:

Zustand	Parameter	Standard	Beschreibung
Zellen aktiv	LOOP_INTERVAL_CELLS_S	120 s (2 min)	Kurzes Intervall für schnelle Reaktion
Keine Zellen	LOOP_INTERVAL_NO_CELLS_S	900 s (15 min)	Langes Intervall spart Ressourcen
Atmosphären-Snap	ATMOSPHERIC_SNAPSHOT_INTERVAL_MIN	90 MIN	Unabhängig von Zellen

■ Beide Intervallwerte sind über runtime_overrides.json ohne Service-Neustart überschreibbar.

21 NEU: LOCAL_TRAINING-Flag

Datei: **config.py** | Parameter: **LOCAL_TRAINING** (bool, default: True)

Dieses Flag bereitet das System für Phase B vor: In Phase B wird ein separater x86-Linux-Rechner das Training übernehmen. Der Pi schaltet dann auf LOCAL_TRAINING=False und synchronisiert fertige Modelle per rsync.

Wert	Verhalten	Phase
True (Default)	Scheduler startet alle Training-Jobs (retrain, rebuild_dataset, Phase A)	Phase A: Pi trainiert selbst
False	Training-Jobs werden übersprungen. Nur accuracy_eval, ai_analysis, extended_cleaner bleiben aktiv.	Phase B: Externer Rechner

Setzen per install.sh oder runtime_overrides.json:

```
bash install.sh --mode=upgrade --no-training # oder in runtime_overrides.json:
{"LOCAL_TRAINING": false}
```

22 NEU: API-Request-Statistik

API-Endpoint: **GET /api/api_health** | Frontend: **Logs.jsx**

Jede externe API-Anfrage (ARSO, Open-Meteo, GeoSphere, Blitzortung, EUMETView) wird in einer lokalen JSON-Datenbank gezählt. Das Admin-Panel zeigt unter **Logs** eine Tabelle mit Requests pro Tag und Schnittstelle sowie Fehlerquoten und letzte Fehler-Meldung.

Spalte	Beschreibung
API-Name	Name der externen Schnittstelle
Requests/Tag	Anzahl Requests heute (wird täglich zurückgesetzt)
Fehler	Anzahl fehlgeschlagener Requests (Timeout, HTTP-Error)
Letzter Fehler	Zeitstempel und Fehler-Meldung des letzten Fehlers
Status	Grün/Gelb/Rot basierend auf Fehlerquote

23 NEU: KI-Analyse Chat (daily_analyzer.py)

Modul: **daily_analyzer.py** | Frontend: **AiSuggestions.jsx** | API: Claude (Anthropic)

Das System kann täglich automatisch den eigenen Zustand analysieren lassen. Dazu wird ein detaillierter Report (Systemmetriken, letzte erkannte Zellen, Quellcode-Auszüge von GitHub) an die Claude-API gesendet und strukturierte Handlungsempfehlungen zurückerhalten.

23.1 Automatische Analyse

Läuft täglich um 06:00 Uhr via Scheduler (cron_hour=6). Analysiert:

- Modell-Genauigkeits-Trends der letzten 24 Stunden
- System-Ressourcen (Disk, RAM)
- API-Fehlerquoten
- Qualität des Trainingsdatensatzes
- Letzte erkannte Sturmzellen (letzten 5 Frames)
- Quellcode der wichtigsten Dateien (von GitHub)

23.2 Interaktiver Chat im Admin-Panel

Unter **/ai-analysis** gibt es zusätzlich einen Chat-Bereich für Fragen an die KI:

- Freie Texteingabe, z.B. 'Warum sind die Vorhersagen für 30-min so ungenau?'
- Modell-Auswahl (Claude Sonnet 4, etc.)
- Toggle: Systemmetriken (letzte 24h) einbeziehen
- Toggle: Quellcode einbeziehen (langsamer, aber genauere Code-Analyse)

23.3 Konfiguration

Parameter	Standard	Beschreibung
enabled	false	Master-Schalter für automatische tägliche Analyse
cron_hour	6	Uhrzeit des automatischen Analyselaufs
model	claude-sonnet-4-20250514	Verwendetes Claude-Modell
max_tokens	1500	Maximale Antwortlänge in Tokens
since_hours	24	Datenfenster für den Report (letzte N Stunden)
save_suggestions	true	Vorschläge als JSON persistieren

■ **Achtung:** Für die KI-Analyse wird ein gültiger Anthropic-API-Key in der .env benötigt:
ANTHROPIC_API_KEY=sk-ant-... Die KI-Analyse verursacht API-Kosten pro Request.

24 Hailo-8 Integration — Phasen-Roadmap

Der Hailo-8 AI-Beschleuniger (26 TOPS, PCIe Gen3) ist physisch montiert und der Treiber (hailo-all) ist installiert. Die Inferenz-Integration erfolgt in zwei Phasen:

Phase	Status	Inhalt
Phase A (aktuell)	In Betrieb	hailo_inference.py als Stub. Alle Modelle laufen auf Pi-CPU. Hailo-8 bleibt ungenutzt — Inferenz
Phase B (geplant)	Ausstehend	Anschaffung x86-Linux-Trainer. U-Net Nowcasting (~2 Mio Parameter) wird trainiert, via DFC
Phase C	Langfristig	Weitere Optimierungen, ggf. zusätzliche Modelle, KI-Analyse vertieft.

Strategische Entscheidung: Statt zweier kleiner CNNs (Cell-Intensity + Cell-Motion) wird ein einzelnes U-Net für komplette Radarbild-Nowcasting eingesetzt. Bei ~2 Mio Parametern wäre dies auf der Pi-CPU nicht im 120s-Takt praktikabel, rechtfertigt aber den Hailo-8 (26 TOPS) vollständig.

■ Hinweis: Während Phase A/B bleibt hailo_inference.py ein Stub der alle Aufrufe transparent auf CPU-Fallback (LightGBM/LSTM) umleitet. Die restliche System-Architektur ändert sich nicht.

25 Konfigurationsreferenz

Alle Parameter werden in **config.py** als Python-Konstanten definiert und können über **runtime_overrides.json** zur Laufzeit überschrieben werden — ohne Service-Neustart. Das Admin-Panel schreibt Änderungen automatisch in runtime_overrides.json.

Parameter	Standard	Bereich
UPSCALE_FACTOR	3.0	Bildverarbeitung
MAX_CONTOUR_DISTANCE	30 px	Tracking
MAX_STATION_DISTANCE_KM	20 km	Tracking
MIN_MOVEMENT_FOR_ARROW_KMH	5 km/h	Darstellung
LOOP_INTERVAL_CELLS_S	120 s	Live-Loop
LOOP_INTERVAL_NO_CELLS_S	900 s	Live-Loop
ATMOSPHERIC_SNAPSHOT_INTERVAL_MIN	180 min	Live-Loop
HAIL_WARN_THRESHOLD	0.45	Warnungen
STATIONARY_RISK_MARKER_THRESHOLD	0.60	Warnungen
GUST_WARN_KMH	60 km/h	Warnungen
HEAVY_RAIN_WARN_MM_PER_H	25 mm/h	Warnungen
MAX_CELL_SPEED_KMH	150 km/h	Plausibilität
MAX_SPEED_CHANGE_PER_CYCLE_KMH	60 km/h	Plausibilität
ML_SEQUENCE_LENGTH	6	ML (LSTM)
ML_FORECAST_HORIZONS_MIN	[10,20,30,40,60]	ML
DATA_RETENTION_DAYS	90	Daten-Rotation
VERIFICATION_TOLERANCE_KM	5 km	Verifikation
AI_ANALYSIS_CONFIG.enabled	false	KI-Analyse
AI_ANALYSIS_CONFIG.cron_hour	6	KI-Analyse
LOCAL_TRAINING	true	Multi-Rechner

■ **Hinweis:** Alle Parameter unter runtime_overrides.json überschreiben die config.py-Defaults. Änderungen über das Admin-Panel sind sofort wirksam.

26 Fehlerbehebung (Troubleshooting)

Problem	Ursache	Lösung
Admin-Panel nicht erreichbar	nginx gestoppt oder Service-Fehler	<code>sudo systemctl status nginx; sudo systemctl restart nginx</code>
Keine Radar-Daten	ARSO-Server nicht erreichbar oder Timeout	API-Adresse unter <code>/logs</code> prüfen; logs mit <code>journalctl -fu wetterprojekt.s</code>
Modelle nicht vorhanden	Erster Start, Training noch nicht gestartet	<code>data/train</code> manuelles Training starten; system läuft im kinematische
Disk kritisch (> 90%)	Trainingsdaten wachsen unbegrenzt	Sofortiger Cleanup: <code>python3 cleanup_old_data.py</code> ; <code>DATA_RETENTION</code>
hailo_inference.py Fehler	Hailo-Treiber Fehler (Phase A: <code>dyvar</code>) — CPU-Fallback ist aktiv. Phase B Hailo-Integration noch au	
SMS nicht gesendet	TWILIO-Credentials fehlen oder <code>env</code> nicht richtig	prüfen; Twilio-Konto und Guthaben prüfen
KI-Analyse schlägt fehl	<code>ANTHROPIC_API_KEY</code> fehlt oder <code>env</code> nicht richtig	KI-Analyse in <code>config.py</code> deaktivieren wenn nicht benötigt
Open-Meteo Limit erreicht	Zu viele Requests (> 10000/Tag)	Bulk-Query in <code>fetch_arome_openmeteo.py</code> aktiviert? TTL erhöhen in AP
Admin-Passwort vergessen	<code>/etc/nginx/.htpasswd</code> fehlt	<code>cat /home/ki-pi/wetterprojekt/.admin_password</code> — oder <code>install.sh --mode</code>
Frontend zeigt leere Seite	Frontend-Build veraltet oder fehlt	<code>cd frontend && npm run build; sudo systemctl restart wetterprojekt-adm</code>

27 Änderungshistorie

Version	Datum	Änderungen
v1.0	2025	Erstveröffentlichung: HSV-Segmentierung, Kalman-Tracking, LSTM + LightGBM, React Admin-Panel (10 S
v1.1	Mai 2026	Phase-A-Erweiterungen: Optical Flow (pysteps), AROME icon_d2 Gitterpunkt, Erweitertes DEM (3 Kacheln

WetterExtended Benutzerhandbuch v1.1 | Stand: Mai 2026 | Repository: github.com/IVOBLA/WetterExtended